

# DESIGN AND ANALYSIS OF ALGORITHM (CS-327)

## LECTURE – 8

### BINARY SEARCH ALGORITHM

**Binary search** is a search algorithm used to find the position of a target value within a sorted array. It works by repeatedly dividing the search interval in half until the target value is found or the interval is empty. The search interval is halved by comparing the target element with the middle value of the search space. The complexity of algorithm is  **$O(\log N)$** .

Example :-

Size of array = N

Key Element = X

|            |       |    |    |    |    |    |    |    |    |     |    |    |    |    |    |    |    |     |     |
|------------|-------|----|----|----|----|----|----|----|----|-----|----|----|----|----|----|----|----|-----|-----|
| Value :-   | 5     | 10 | 15 | 20 | 25 | 30 | 35 | 40 | 45 | 50  | 55 | 60 | 65 | 70 | 75 | 90 | 95 | 100 | 105 |
| Location:- | 0     | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9   | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17  | 18  |
|            | ↓     |    |    |    |    |    |    |    |    | ↓   |    |    |    |    |    |    |    |     | ↓   |
|            | Start |    |    |    |    |    |    |    |    | Mid |    |    |    |    |    |    |    |     | End |

Mid =(start+end)/2 =9

if(arr[mid]==x)

{

return(mid)

}

if(start==end)

{

Return -1;

}

if(arr[mid]>x)

{

return B-search(arr,x,start,mid-1);

}

if(arr[mid]<x)

{

Return B-search(arr,x,mid+1,end); }

| Binary Search Algorithm                         | Time Complexity                                                               |
|-------------------------------------------------|-------------------------------------------------------------------------------|
| int B-search(int arr[],int x,int start,int end) | $T(N) = C + T(N/2)$ -----(i)                                                  |
| {                                               | $T(N/2) = C + T(N/4)$ -----(ii)                                               |
| int mid= (start+end)/2                          | Substitute eq(ii) in eq(i)                                                    |
| if(arr[mid]==x)                                 | $T(N) = T(N/4) + 2C$ ----- (iii)                                              |
| {                                               | $T(N/4) = C + T(N/8)$ ----- (iv)                                              |
| Return mid;                                     | Substitute eq(iv) in eq(iii)                                                  |
| }                                               | $T(N) = T(N/8) + 3C$ ----- (v)                                                |
| If(start==end)                                  | Pattern we found from above                                                   |
| {                                               | $T(N) = T(N/2^i) + iC$ ----- (vi)                                             |
| Return -1;                                      | If we keep on solving or breaking $N/2^i$ , so it will reach 1 element array. |
| }                                               |                                                                               |
| if(arr[mid]>x)                                  | $T(N/2^i) = T(1)$                                                             |
| {                                               | $N/2^i = 1$                                                                   |
| return B-search(arr,x,start,mid-1);             | $N = 2^i$                                                                     |
| }                                               | Take log2 on both sides                                                       |
| if(arr[mid]<x)                                  | $\log_2 N = \log_2 2^i$ ( $\therefore \log_2 2 = 1$ or $\log_{10} 10 = 1$ )   |
| {                                               | $\log_2 N = i$ ----- (vii)                                                    |
| Return B-search(arr,x,mid+1,end);               | Substitute eq (vii) in eq(vi)                                                 |
| }                                               | $T(N) = T(N/2^{\log_2 N}) + C \log_2 N$<br>( $\therefore 2^{\log_2 N} = N$ )  |
| }                                               | $T(N) = T(N/N) + C \log_2 N$                                                  |
|                                                 | $T(N) = T(1) + C \log_2 N$                                                    |
|                                                 | $T(N) = K + C \log_2 N$                                                       |
|                                                 | $T(N)$ is $O(\log_2 N)$                                                       |

Prove that :-  $2^{\log_2 N} = N$

**Solution** :-

$$\text{Let } y = 2^{\log_2 N}$$

taking  $\log_2$  on both sides

$$\log_2 y = \log_2 2^{\log_2 N}$$

$$\log_2 y = 1 \cdot \log_2 N \quad (\because \log_2 2 = 1 \text{ or } \log_{10} 10 = 1)$$

$$\cancel{\log_2} y = \cancel{\log_2} N$$

$$y = N$$

$$2^{\log_2 N} = N \text{ Proved}$$